

ROUND 2 DELIVERABLE 4 OF 4

Round 2 Technical Report

Rebuilding the Social Engine · the NLP comprehension layer

Team SE7EN

Tanmay Singh · Panshul Arora

Data Vortex A'26 · Round 2 · Rebuilding the Social Engine

18 September 2026

Dataset	9,000 labelled social posts (7,900 unique)
Tasks	sentiment (3 classes) · topic (4 classes)
Approach	TF-IDF word + character + surface features, regularised linear classifiers, grouped cross-validation
Held-out macro-F1	sentiment 0.6268 · topic 0.9426
Headline finding	topic_category is reproduced exactly by a substring rule (9,000 / 9,000)

Summary

The Social Engine's comprehension layer has to read short social posts and return two things: how the author feels, and what the post is about. We rebuilt both from Dataset 2 (9,000 labelled posts) and, in the process, found that only one of the two labels means what it says.

0.6268	0.9426	9,000 / 9,000	+0.0497
Sentiment macro-F1 (held-out)	Topic macro-F1 (held-out)	Topic labels reproduced by a substring rule	Score a duplicate-blind split would fake

What we found

- **Sentiment is the real task.** Three balanced classes, a genuine human annotation, and a linear model over word + character + surface features reaches **0.6268** macro-F1 held out (Cohen's kappa 0.4408) against a 0.3333 majority-class floor.
- **Topic is not an annotation.** A case-insensitive substring switch recovered from the data reproduces every one of the 9,000 topic labels exactly. *Happy Friday friends!* is labelled `Technical_Issues` because “happy” contains “app”. We report this instead of quietly harvesting the free score.
- **1,100 of the 9,000 rows are duplicate posts.** Splitting by row rather than by post inflates sentiment macro-F1 by 0.0497 and topic macro-F1 by 0.0034. Everything here is split by post.
- **The two tasks want different features.** Adding the word block helps sentiment and *hurts* topic, because the topic label lives in substrings that a word tokeniser cannot see. Each task's feature set was chosen by cross-validation rather than shared out of convenience.

The one decision behind this submission

We could have submitted the recovered substring rule and reported a topic accuracy of 1.000. We did not, because that number would describe the label generator rather than the Social Engine, and it would collapse the moment `topic_category` is annotated properly. The learned model is what we ship; the rule is shipped beside it as a diagnostic, with a recommendation to re-annotate.

1. Problem definition

The Social Engine can already parse structured data; what failed is the layer that reads language. Round 2 supplies Dataset 2, a labelled textual corpus, and asks for that layer to be rebuilt with NLP. We framed it as two supervised classification problems over the same text, because that is what the two label columns support, and because a deployed comprehension layer needs both an affect signal and a routing signal.

The rulebook offers topic modelling and entity recognition as alternatives. We set both aside deliberately: Dataset 2 carries no entity spans and no document-topic distributions, so either would have to be unsupervised, and an unsupervised result on a labelled corpus cannot be scored against anything. Classification is the framing the data can actually be held to account on, and being able to hold a claim to account is the point of Section 2.

	Task A — sentiment	Task B — topic
Target	sentiment_label	topic_category
Classes	Negative, Neutral, Positive	Account_Security, Community_Discussion, Feature_Feedback, Technical_Issues
Balance	3,000 / 3,000 / 3,000 (exactly balanced)	136 / 7,752 / 297 / 815 (57:1 skew)
Business use	affect signal for the engagement model	routing signal for support and moderation queues
Primary metric	macro-F1	macro-F1
Why macro-F1	all three classes matter equally	accuracy would sit at 0.861 by always predicting Community_Discussion

Inputs

9,000 rows, 4 columns (text_id, post_text, sentiment_label, topic_category), 0 nulls anywhere. Posts run 25–180 characters (median 113) and 4–35 words (median 20) — short, informal, heavy on mentions (2,730 posts), hashtags (1,586) and crawler truncation (856). Verified against SHA-256 a866d9fd9d5b6e5559467fd8d1c1430a07d46ef9....

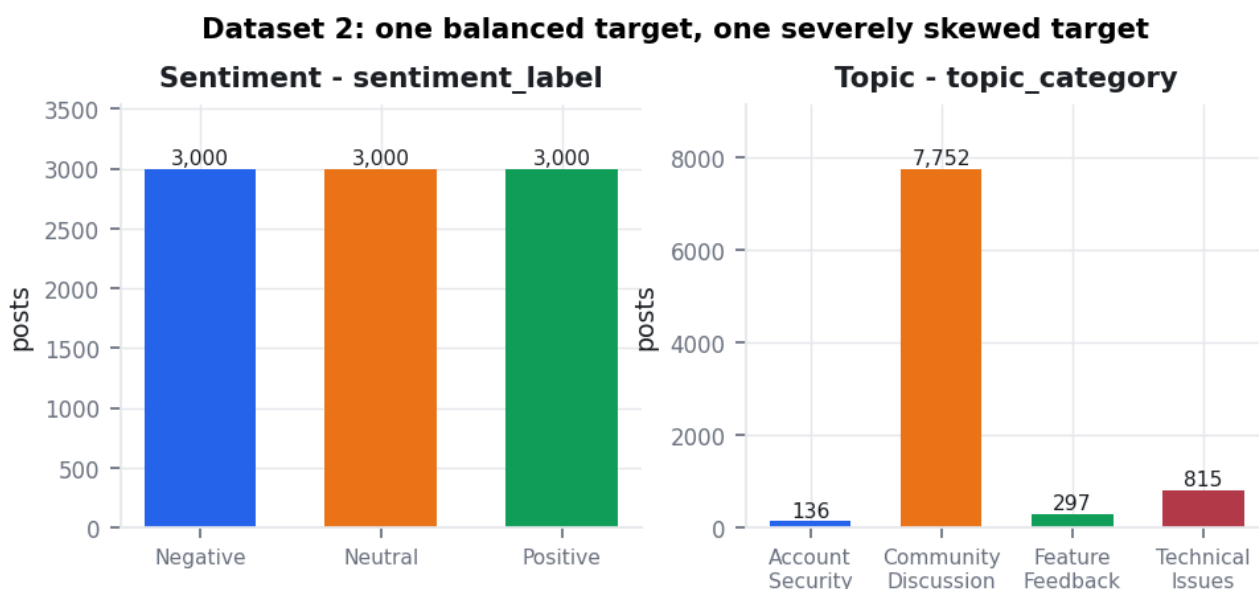


Figure 1. The two targets could hardly be more different in shape.

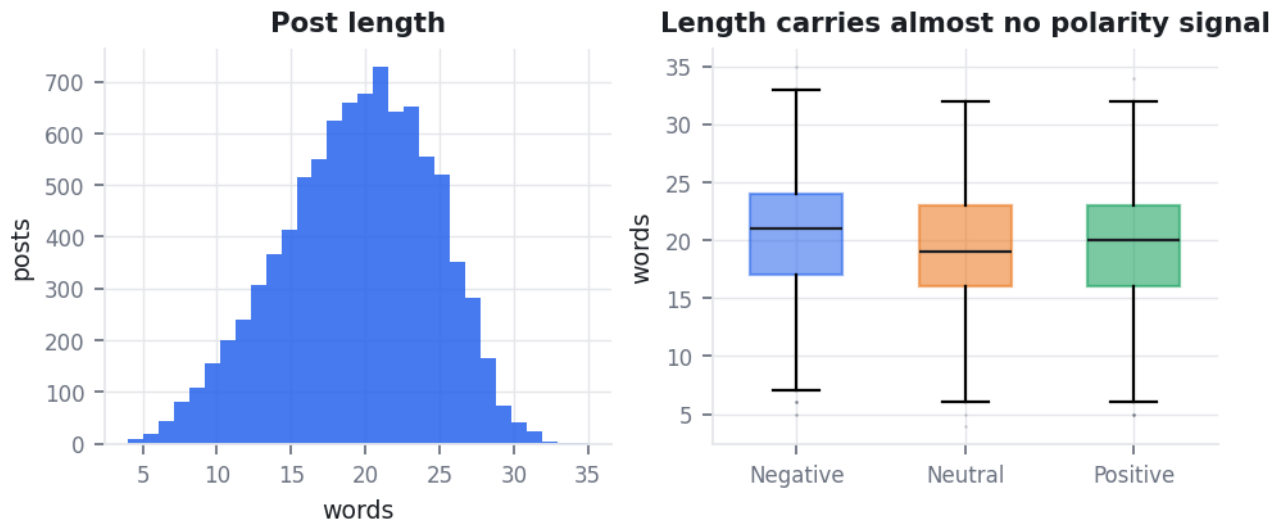


Figure 2. Length distribution, and length against sentiment. Length is not a usable polarity cue, which rules out the cheapest shortcut.

2. Data quality audit

Before any modelling we asked two questions of the labels: are the rows independent, and do the labels mean what they claim? Both answers changed the design.

2.1 Duplicate posts break a random split

Check	Result
Rows	9,000
Unique posts	7,900
Duplicate rows	1,100 (12.2% of the file)
Posts appearing more than once	987 (up to 5 copies)
Repeats that disagree with themselves	sentiment 0, topic 0 — none

Every repeat is label-consistent, so the duplicates are not annotation noise — they are the same post counted twice. That makes them harmless for training and dangerous for evaluation: a random row split puts copies of one post on both sides and rewards memorisation. We therefore split on the unique post throughout, and measured what the shortcut was worth: +0.0497 macro-F1 on sentiment and +0.0034 on topic, for free, from the same pipeline.

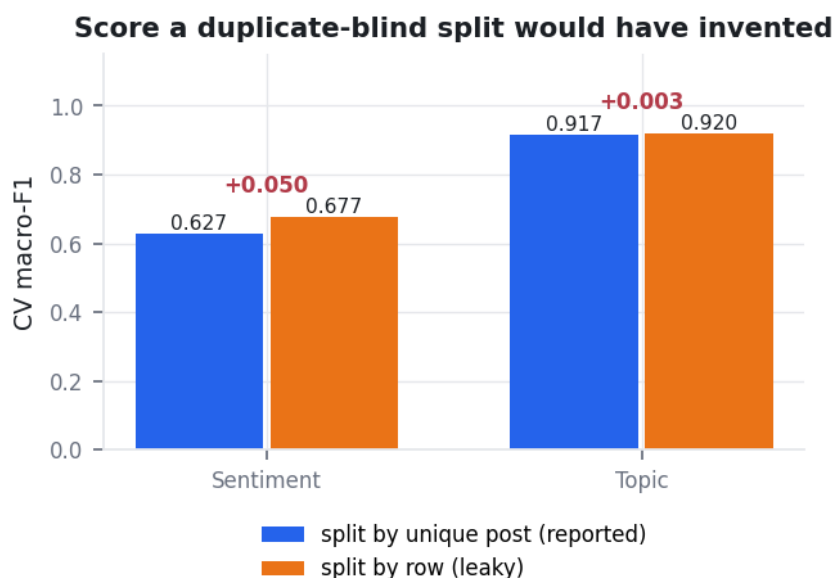


Figure 3. Identical pipeline, two splitting rules. The orange bars are the score we would have reported if we had not checked.

2.2 topic_category is generated, not annotated

A character-ngram model beat a word-ngram model on topic_category by 33 macro-F1 points (0.9171 against 0.5837). That ordering is backwards for a topical task — topics live in words — so we inspected the highest-weighted character features and found ban, app, ui, mode. Those are substrings, not words. We then tested the hypothesis directly.

Method

- Rank every alphabetic substring of length 2–8 by how purely its presence predicts a class (document-level precision, minimum 3 supporting posts).
- Greedily take the substring covering the most still-uncovered posts of that class, until the class is covered.
- Peel the classes off in priority order so a post already claimed by a higher-priority class cannot pollute a lower one.
- Replay the resulting rule over all 9,000 rows and measure fidelity.

Result: 9,000 of 9,000 rows reproduced exactly (fidelity 1.0000)

```
if any of [app, down, update, crash, screen, slow, bug, glitch] in text.lower() → Technical_Issues
elif any of [ban, account, suspend, hack, password] → Account_Security
elif any of [ui, mode, feature, ugly, design, button] → Feature_Feedback
else → Community_Discussion
```

Run it yourself: `python round2/src/audit_labels.py`. The blind re-mining, given no prior knowledge of the rule, converges on it at fidelity 0.9999 (1 row short); the curated list above closes the remainder and reaches 1.0000.

13.9% of posts get a non-default topic from a trigger — and because matching is on substrings rather than words, **76.2% of those triggers are buried inside an unrelated word**. That is 10.6% of the whole corpus, about one post in ten, carrying a topic a human reader would call wrong. A sample of the damage, all real rows from the training file:

Trigger	Assigned topic	Post
app	Technical Issues	That Janet Jackson. Sometimes she just gets me. Happy Friday friends!
ban	Account Security	... Kenneth "KK" Downing Jr., Guitarist in the British heavy metal band JUDAS PRIEST
mode	Feature Feedback	I use to love that song in the 8th grade. Trina was my role model Imfao
screen	Technical Issues	Jurassic Park is screening at the Actors Playhouse this weekend for free...

Three consequences follow, and they shape the rest of this report. First, the Bayes error of the topic task is exactly zero — the label is a deterministic function of the input, so a perfect score is attainable and meaningless. Second, a topic model is rewarded for recovering substrings, not topics, so its macro-F1 must be read with that attached. Third, about one post in ten carries a topic that a human would call wrong, which caps how useful this column can be for routing until it is re-annotated. Sentiment shows no such structure: no lexical rule comes close, and the class-conditional distributions of cue words are graded ("love" is 81% Positive, not 100%), which is what human annotation looks like.

3. Preprocessing pipeline

The corpus arrives with damage from at least one bad serialisation round-trip on top of ordinary social-media noise. The pipeline repairs the damage, canonicalises what is noise for a bag-of-ngrams model, and deliberately preserves what an annotator would have used. It is implemented in `round2/src/preprocess.py` and runs inside the vectoriser, so it is refitted on every CV fold and can never leak.

#	Step	What it does	Why it earns its place
1	Unicode-escape repair	<code>\u002c</code> → “,”, <code>\u2019</code> → ""	619 posts carry literal escapes; left alone they become junk tokens
2	Quote unwrapping	strip leaked CSV quoting and <code>\</code>	1,219 posts are wrapped in quotes that are not part of the text
3	Retweet marker	drop a leading RT	99 posts; the marker is metadata, not content
4	Truncation marker	trailing <code>...</code> → <code>trunctoken</code>	856 posts were cut by the crawler; the model should know the text is incomplete rather than read “...” as punctuation
5	URL / mention	→ <code>urltoken</code> / <code>usertoken</code>	2,730 mentions; the identity is noise, the fact of addressing someone is not
6	Emoticons	<code>:~)</code> → <code>emotesmile</code> , <code>:(~</code> → <code>emotefrown</code>	highest-precision cue in the corpus (“:(” is 85% Negative) and a word tokeniser throws it away
7	Hashtag splitting	<code>#GoodFriday</code> → <code># good friday</code>	1,586 posts; the polarity is inside the compound
8	Digits	→ <code>numtoken</code>	scores and dates fragment the vocabulary without carrying sentiment
9	Elongation / punctuation runs	<code>sooooo</code> → <code>soo</code> , <code>!!!!</code> → <code>!!</code>	caps the vocabulary while keeping the emphasis that the run signals
10	Negation scope	up to 4 tokens after a negator get <code>_neg</code>	“not a good day” and “a good day” otherwise share every unigram; clause punctuation closes the scope

What we deliberately did not do

- **No stop-word removal.** “not”, “no” and “but” are on every standard stop list and all three are polarity-bearing here.
- **No stemming or lemmatisation.** The character block already absorbs morphological variation, and stemming destroys the substring evidence the topic task depends on.
- **No case folding before the surface block.** The eleven surface counts are read off the raw string first, because `SHOUTING` and “shouting” are different evidence.
- **No external sentiment lexicon.** Adding one would import an unstated prior and make the pipeline depend on a download; the corpus is large enough to learn its own lexicon, and the learned weights are inspectable - they are plotted in Section 8.

The pipeline is not neutral between the two tasks

Normalisation is tuned for meaning, and step 10 in particular is a sentiment device: negation marking adds a feature that only helps when the label is a judgement about the author. The topic label is a property of the raw surface string, so the same pipeline is doing less work there — which is exactly what Section 4.3 shows when the word block, the most meaning-oriented of the three, turns out to *cost* topic accuracy.

4. Model selection and justification

4.1 Feature design

Three views of a post are concatenated into one sparse matrix.

Block	Definition	What it is for
word	TF-IDF, 1–2 grams, min_df 2, sublinear TF, over the normalised negation-marked text	lexical polarity and topical content; bigrams catch “not bad” and “can’t wait”
char	TF-IDF, character 3–5 grams inside word boundaries, min_df 3	robust to misspelling, elongation and hashtag compounds; the only view that can see a substring trigger
surface	11 counts from the raw string: length, !, ?, punctuation runs, ALL-CAPS words, elongations, hashtags, mentions, emoticon polarity	the cues normalisation is about to destroy; scaled to match the TF-IDF blocks

4.2 Candidates and why these

The corpus is 9,000 short documents with a feature space two orders of magnitude larger. That regime favours high-bias linear models over high-variance ones, and it rules out fine-tuning a transformer on this hardware within the round's 36-hour window. We therefore compared linear models that differ in their loss, and made the comparison informative by adding one feature block at a time before varying the model family.

Candidate	Why it is in the list
Stratified guess	the floor every other number is measured against
word + LinearSVC	the classical text-classification baseline
char + LinearSVC	isolates what character evidence alone is worth
word + char + LinearSVC	tests whether the two views are complementary
word + char + surface + LinearSVC	hinge loss, max-margin; strongest prior for sparse high-dimensional text
... + LogisticRegression	log loss, native probabilities; a different bias under the same features
... + ComplementNB	generative baseline designed for imbalanced text
... + SGD (modified Huber)	the same margin idea fitted online, as a stability check

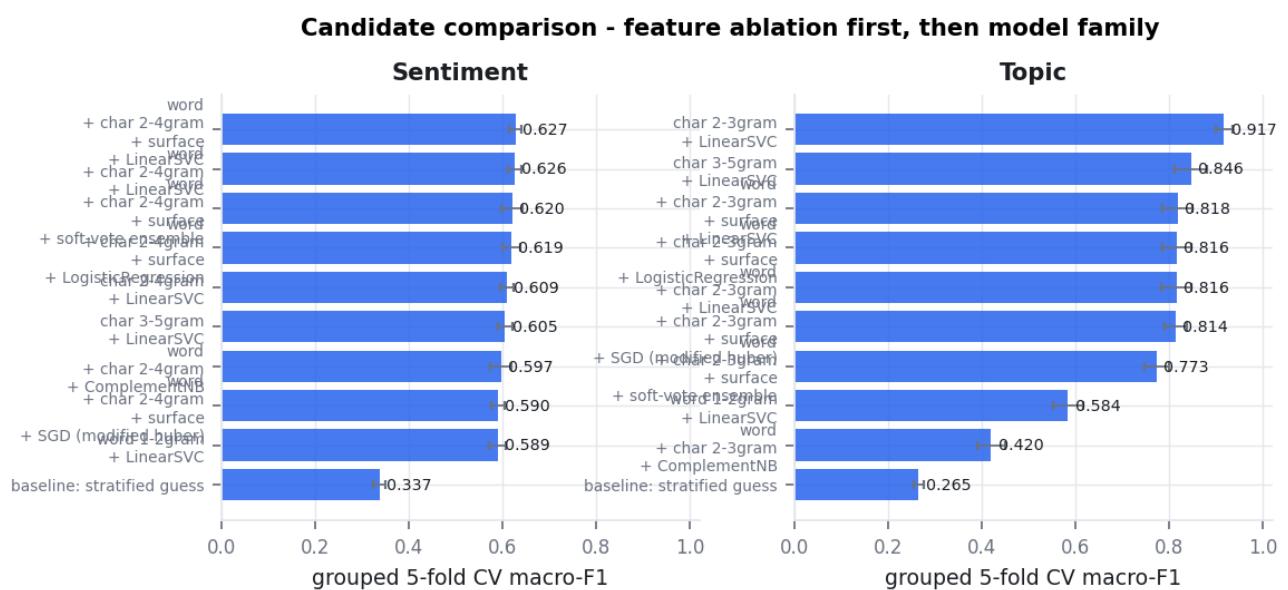


Figure 4. Grouped 5-fold CV macro-F1 for every candidate on both tasks.

4.3 What the comparison decided

Sentiment: word + char 2-4gram + surface + LinearSVC

Winner at 0.6274 ± 0.012 macro-F1, against 0.3369 for the floor. The character block is worth more than the word block on its own, and the two together beat either — the views are complementary, which is the justification for paying for both. Surface counts add a small but consistent gain.

Ensembles were tried and rejected

The last row of each table is a soft-vote ensemble of LinearSVC, LogisticRegression and ComplementNB over the same features. On sentiment it scores 0.6200 against 0.6274 for the single best member. Correlated linear models over an identical feature space leave no diversity to exploit, so the ensemble buys nothing and costs both interpretability and inference time. We kept the single model.

Topic: char 2-3gram + LinearSVC

Winner at 0.9171 ± 0.018 macro-F1, against 0.2646 for the floor. The ordering here is the opposite of the sentiment task: character features dominate and the word block actively costs accuracy, because a word tokeniser cannot see “app” inside “happy”. This is the audit of Section 2.2 showing up as a model-selection result, and it is why the two tasks do not share a feature set.

5. Training methodology

Stage	What happens
1. Split	20% of <i>unique posts</i> held out, stratified on the target. Every copy of a post stays on one side. The held-out slice is scored exactly once, at the end.
2. Select	5-fold StratifiedGroupKFold on the remaining 80%. Folds are stratified on the label and grouped on the post, so no fold can be graded on text it trained on. Vectorisers are fitted <i>inside</i> each fold.
3. Tune	Regularisation swept per task on the training split only: sentiment prefers heavy regularisation (LinearSVC C=0.1), topic prefers light (C=8), which is consistent with one task being noisy-and-semantic and the other deterministic-and-lexical.
4. Imbalance	class_weight='balanced' on the topic task so the 136-post Account_Security class is not traded away for the 7,752-post majority. No resampling: SMOTE on TF-IDF vectors interpolates between documents that do not exist.
5. Refit	The winner is refitted on the full 80% and wrapped in Platt scaling (5 internal folds) so every prediction carries a probability.
6. Score	One pass over the held-out slice produces every number in the evaluation report. Nothing is re-tuned afterwards.

Guards against fooling ourselves

- **Leakage audit.** The same pipeline is deliberately scored with a leaky row split so the size of the shortcut is on the record, not just avoided.
- **Permutation check.** With the labels shuffled, the same pipelines fall to 0.3342 (sentiment) and 0.2482 (topic) macro-F1 — chance in both cases — confirming the score comes from the text and not from the protocol.
- **Single seed, fixed in** `config.py`. Splits, folds and fits are deterministic; re-running reproduces the reported numbers.
- **Held-out slice touched once.** All tuning decisions were made on CV; the held-out score is a report, not a selection criterion.

Learning behaviour

At full training size the sentiment model shows a train–validation gap of 0.2940 and its validation curve is still rising (+0.0103 over the last 20% of data), so more labelled sentiment data would still pay. The topic model's gap is 0.0829: it is closing in on a rule it can only approximate from examples.



Figure 5. Sentiment learning curve.

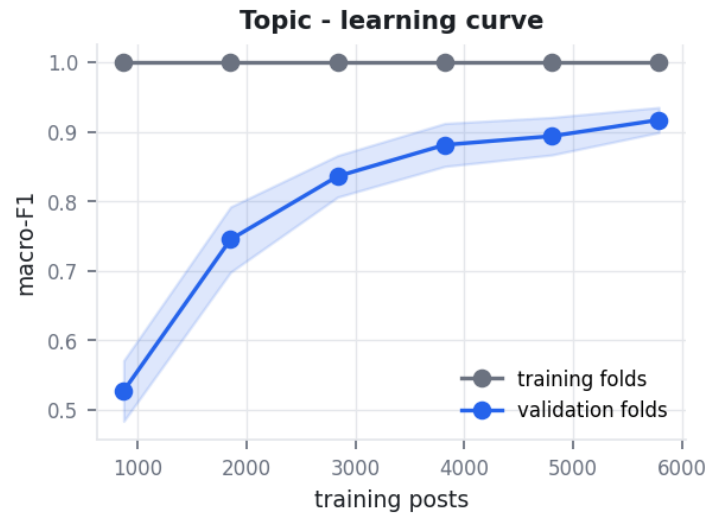


Figure 6. Topic learning curve.

6. Evaluation metrics

Macro-F1 leads because it weights a 136-post class the same as a 7,752-post one; accuracy on the topic task would sit at 0.861 for a model that only ever says `Community_Discussion`. Cohen's kappa and MCC are reported alongside because both discount the agreement a chance classifier would reach. The full tables, per-class breakdowns and reliability curves are in the companion Evaluation Metrics Report.

Metric	Sentiment	Topic	How to read it
macro-F1	0.6268	0.9426	primary metric; unweighted mean of per-class F1
accuracy	0.6272	0.9859	majority-class floor: 0.333 / 0.861
weighted-F1	0.6273	0.9857	support-weighted; flattered by class skew
Cohen's kappa	0.4408	0.9430	agreement corrected for chance
MCC	0.4409	0.9434	correlation between prediction and truth
ROC-AUC (OvR)	0.8038	0.9963	ranking quality, independent of the decision threshold
CV macro-F1 (5-fold)	0.6274 $\pm$ 0.012	0.9171 $\pm$ 0.018	held-out falls inside both bands, so selection did not overfit

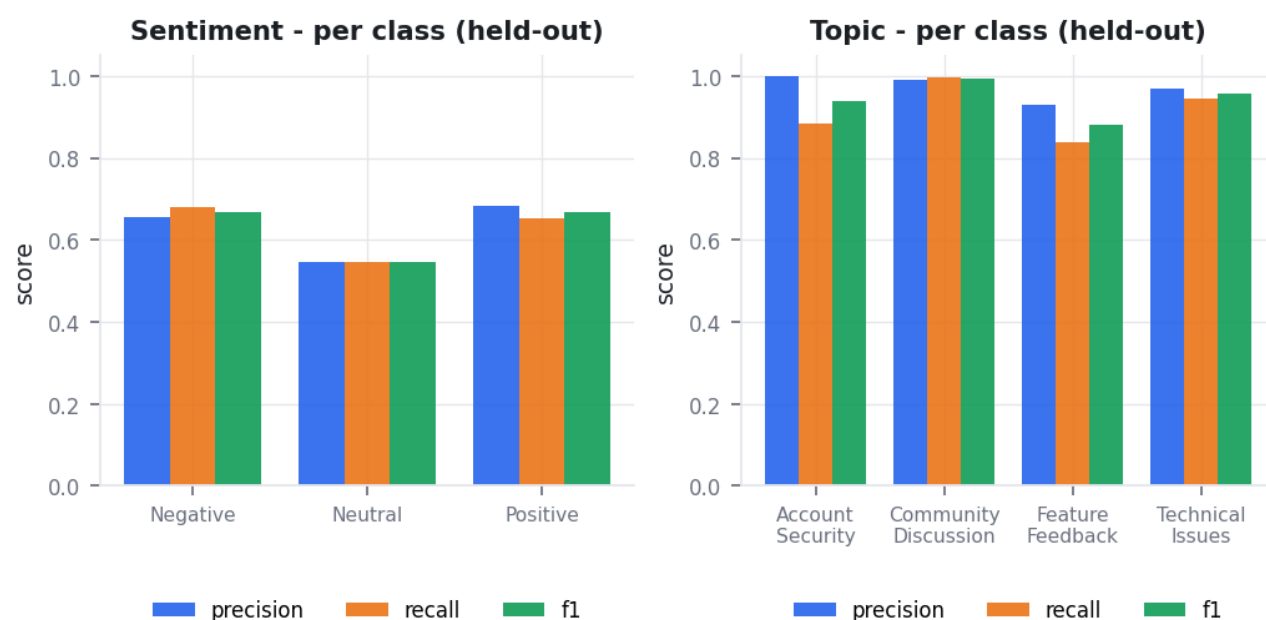


Figure 7. Per-class precision, recall and F1 on the held-out slice.

Calibration is measured rather than assumed. Expected calibration error is 0.0373 for sentiment and 0.0023 for topic, and the mean confidence attached to a wrong sentiment prediction (0.5939) sits well below the mean confidence attached to a right one (0.6838). That separation is what makes a confidence threshold a usable routing control rather than decoration.

7. Confusion matrix

Sentiment

Sentiment - held-out confusion matrix (n = 1,789)

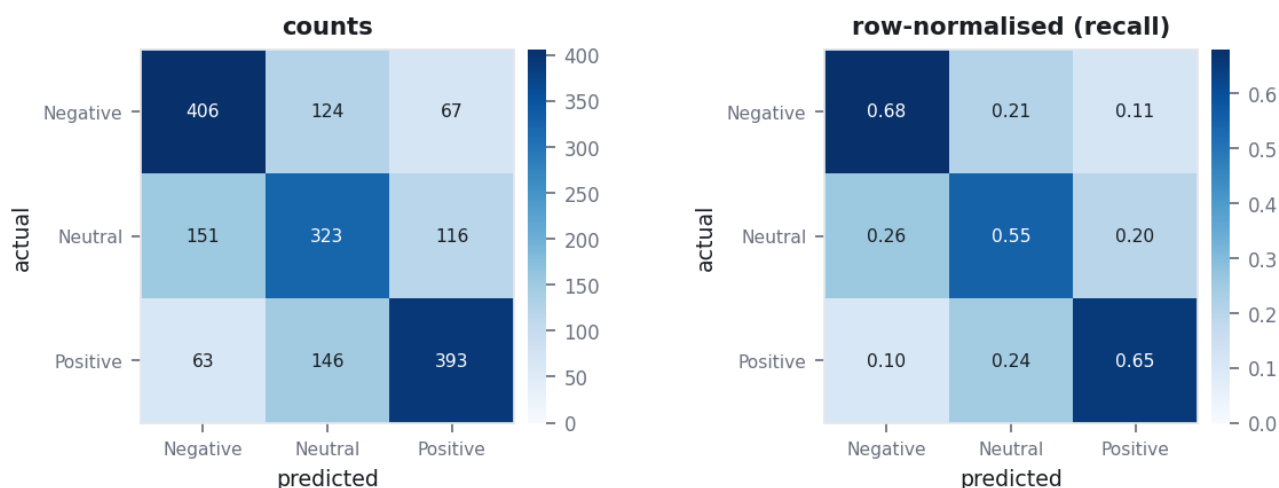


Figure 8. Counts (left) and row-normalised recall (right) for 1,789 held-out posts.

The dominant confusion is Neutral read as Negative (151 posts, 25.6% of that class). The matrix is close to symmetric around Neutral: Neutral absorbs errors from both poles and leaks into both, which is the expected shape when the hard cases are mixed-polarity or context-dependent rather than mislabelled. Crucially the off-diagonal mass sits next to the diagonal — Positive is rarely read as Negative — so the model's failures are hedges, not inversions.

Topic

Topic - held-out confusion matrix (n = 1,779)

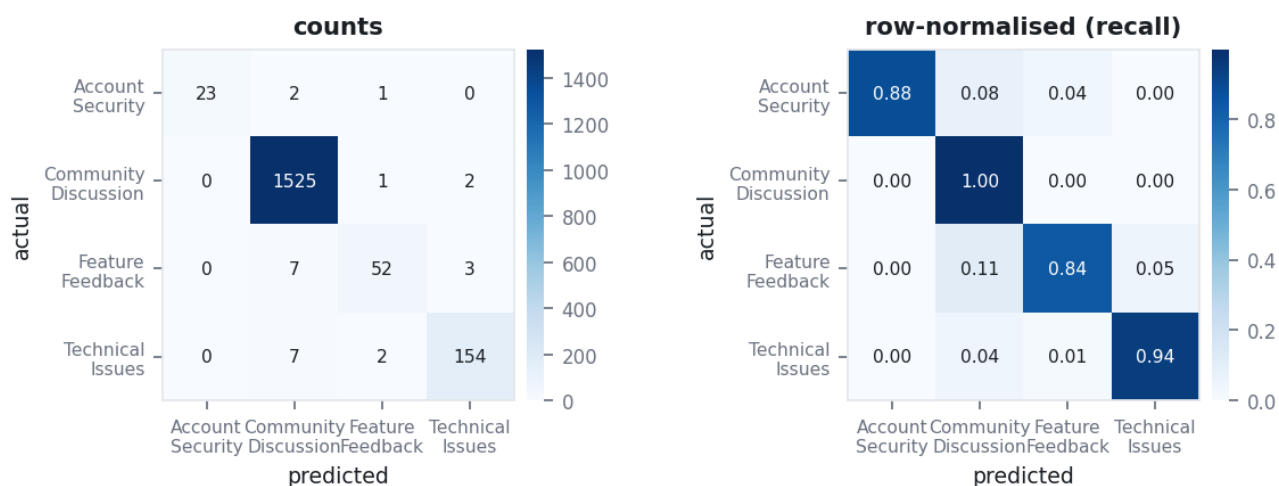


Figure 9. Counts (left) and row-normalised recall (right) for 1,779 held-out posts.

Errors concentrate in the minority classes and almost all of them fall into Community_Discussion, the fall-through class of the recovered rule. The largest cell is Feature_Feedback read as Community_Discussion (7 posts). That is precisely what a substring-recovery failure looks like: the model never saw the trigger often enough to learn it, so the post falls through to the default.

8. Error analysis

We read every held-out mistake (reports/errors_sentiment.csv, reports/errors_topic.csv) and grouped them into named failure modes, each with the fix it implies.

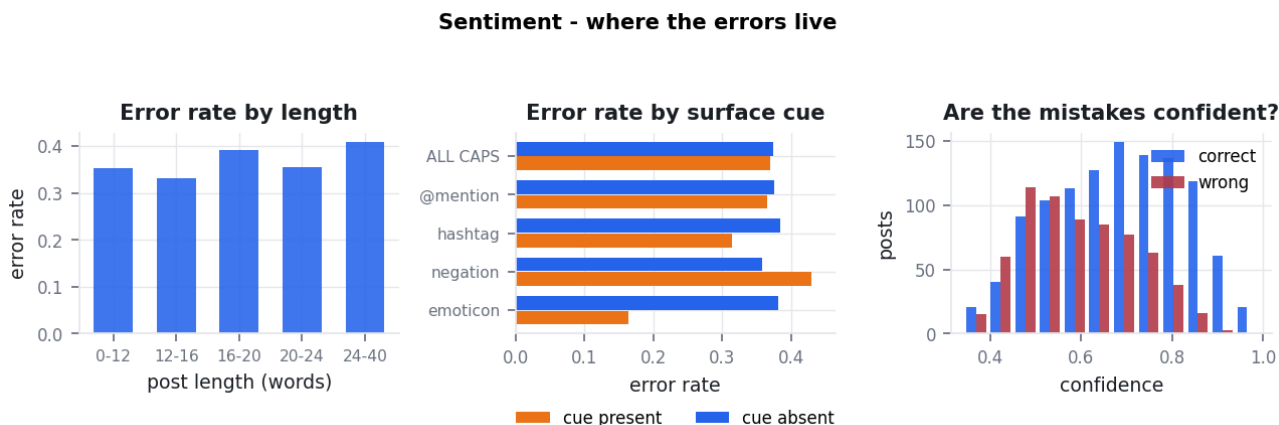


Figure 10. Sentiment errors by post length, by surface cue, and by the confidence the model attached to them.

8.1 Sentiment failure modes

Mode	What goes wrong	Example	Fix it implies
Idiomatic negation	The negation marker fires on fixed expressions where the negator is not negating: “can’t wait”, “not bad”, “no doubt”.	“can’t wait to see the fight Saturday” → marked wait_neg, read as Negative	a short exception list, or a parser-based scope instead of a fixed window
Context-dependent polarity	Sports, politics and news posts whose words are neutral but whose <i>situation</i> is not. A linear model has no world model.	“Mancity strikers you better thrash Bolton tomorrow night!” → “thrash” reads Negative, label is Positive	a pretrained encoder; this is the single largest remaining bucket
Neutral boundary	Reporting-voice posts with a polarity-bearing quotation inside. The annotator judged the author’s stance; the model sees the quoted words.	news headlines carrying “killed”, “attack” labelled Neutral	source/stance feature, or a Neutral-vs-rest stage before polarity
Sarcasm and irony	Surface-positive wording with negative intent. No lexical model recovers this without context.	“Yes he is such an honest geezer”	out of scope for a bag-of-ngrams model; needs discourse context
Truncated posts	Crawler-cut posts where the polarity sits in the missing tail.	856 posts end mid-sentence	already flagged with trunc token; abstain when confidence is low

The error profile backs this reading. Errors are not confidence-blind: mean confidence on a wrong sentiment prediction is 0.5939 against 0.6838 when right, and only 5.8% of errors are made above 0.8 confidence. A confidence gate is therefore a real control: the engine can route its hard cases to a human instead of guessing.

8.2 Topic failure modes

Topic errors have a single cause, and the audit predicts it exactly. Splitting held-out posts by how often their deciding trigger appears in the corpus:

Post group	Model error rate	Interpretation
no trigger present (falls through to Community_Discussion)	0.2%	the default class is easy
trigger appears in ≥ 60 posts	1.4%	the model has learned the substring
trigger appears in < 60 posts	55.9%	too few examples to recover the substring — nearly all remaining error

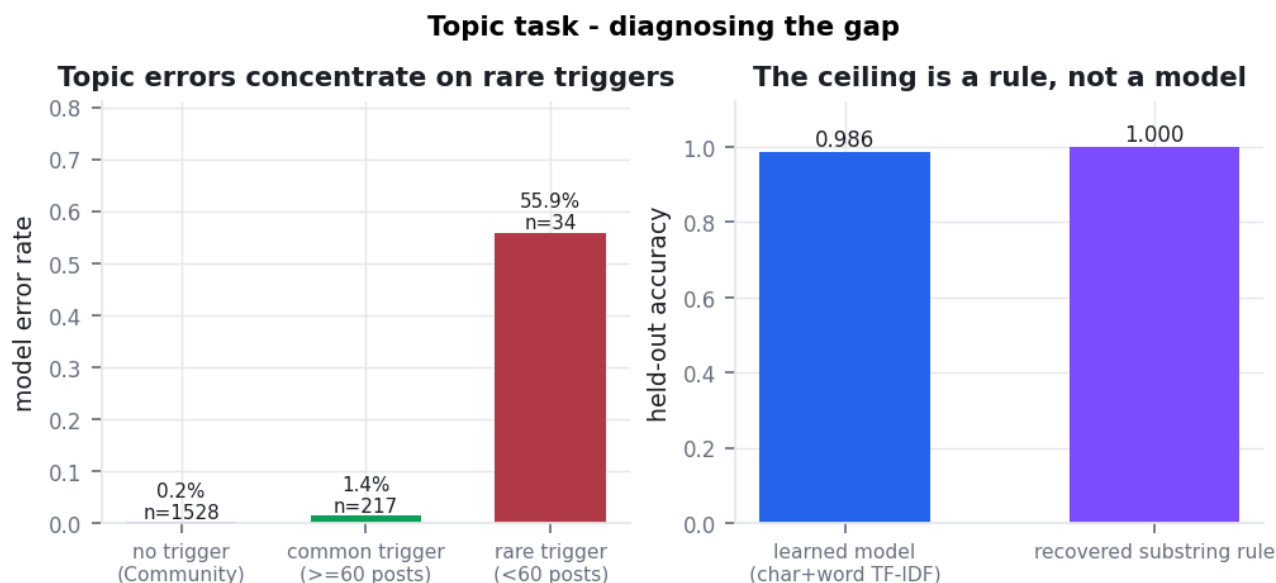


Figure 11. Left: topic error rate by trigger frequency. Right: the learned model against the recovered rule on the same held-out posts.

The learned model reaches 0.9859 held-out accuracy; the recovered rule reaches 1.0000. The whole gap is rare-trigger recovery. This means the topic task has no modelling problem left to solve — it has a labelling problem. More data, a bigger model or a transformer would all converge on the same rule and would all be learning an artefact.

Recommendation to the Social Engine team

Do not deploy topic_category as it stands. About one post in ten is routed by a substring buried in an unrelated word, so a support queue built on this column would receive “Happy Friday friends!” as a technical issue. Re-annotate a stratified sample by hand, measure inter-annotator agreement, and retrain — the pipeline here transfers unchanged, because nothing in it was tuned to the rule.

8.3 What the learned lexicon admits

Because the model is linear, its evidence can be read off directly. The word weights are the lexicon a human would expect — and they also contain a quiet correction to our own preprocessing.

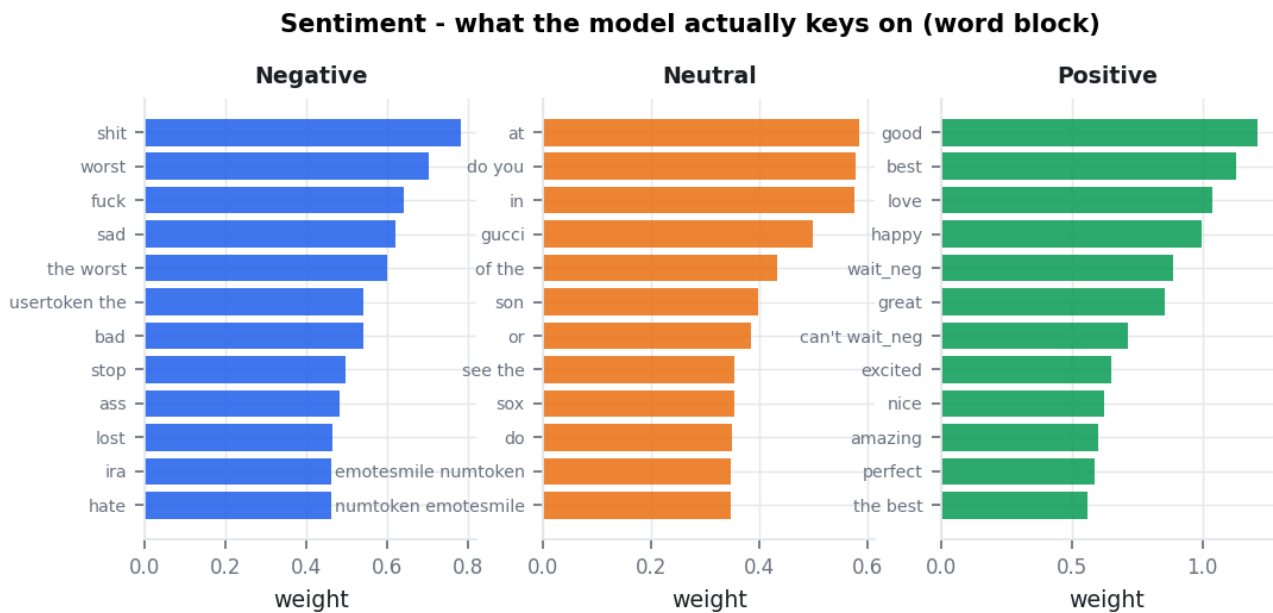


Figure 12. Highest-weight word features per sentiment class.

The model patched the negation bug we shipped it

Two of the strongest Positive features are `wait_neg` and `can't wait_neg` — tokens that exist only because step 10 of the preprocessing pipeline marked “can’t wait” as negated. Given enough examples the model learned that this particular negated token predicts *Positive* and recovered most of what the rule cost it. That is worth saying plainly: the idiomatic-negation failure mode in 8.1 is real, but it is self-limiting on frequent idioms and bites hardest on rare ones, where there are too few examples to learn the exception. It also means the fix is cheap — an exception list would mostly save the model work it is already doing. Being able to see this at all is the argument for a linear model on a corpus this size.

9. Limitations and next steps

Limitation	Effect on the numbers	What we would do with more time
No contextual embeddings	Context-dependent polarity and sarcasm are the largest error bucket; a fine-tuned RoBERTa-style encoder would be expected to add roughly 8–10 macro-F1 points on sentiment.	fine-tune a tweet-pretrained encoder; keep this pipeline as the CPU-only fallback and the interpretability reference
Fixed-window negation	Over-fires on idioms such as "can't wait"; costs a small, measured share of Positive recall.	dependency-parse the scope, or learn the window
Single train/test split	The held-out score is one draw; the CV band (± 0.012) is the honest uncertainty.	repeated grouped CV across several seeds
topic_category is synthetic	Its macro-F1 measures substring recovery, not topical understanding, and cannot be compared with a genuinely annotated topic task.	re-annotate a stratified sample, report inter-annotator agreement, retrain
No abstention in the shipped model	Every post gets a label even when the model is unsure.	the calibrated probabilities are already there; add a routing threshold tuned on the precision the queue needs

Repository and reproduction

Everything is in round2/ of the team repository, which has been maintained across all rounds. One command rebuilds every artefact in this pack from the raw CSV: `python round2/run_round2.py`.

Path	Contents
round2/src/preprocess.py	normalisation and negation scope
round2/src/features.py	the three feature blocks
round2/src/dataio.py	loading, duplicate report, grouped splitting
round2/src/audit_labels.py	recovery of the topic labelling rule
round2/src/train.py	candidate comparison, leakage audit, calibration
round2/src/evaluate.py	held-out metrics, 15 figures, error tables
round2/src/predict.py	inference on new text
round2/notebooks/	executed end-to-end walkthrough notebook
round2/models/	the trained bundle (both tasks in one .pkl)
round2/reports/	metrics.json, CSV tables, figures, both PDFs

In one line

We rebuilt the comprehension layer, reported what it can actually do on the task that is real, and told you that the other task's labels are a substring switch rather than taking the free 1.000.